The Non-Relational Blind Spot: Evaluating LLMs on Multi-Table Numerical Reasoning

Anonymous ACL submission

Abstract

Large language models (LLMs) are increasingly used for tabular question answering, yet most evaluations assume clean, relational schemas. This leaves a gap for real-world analytical queries, where evidence may be distributed across multiple non-relational tables and encoded through layout, noise, units, or implicit cross-table links. Existing relational datasets are difficult to adapt to this setting because complex perturbations can break table validity and answer verification. We introduce GRADINO, a from-scratch benchmark generator for controlled numerical QA evaluation over multiple non-relational tables. GRADINO synthesizes a latent relational source for executable SQL supervision, then evaluates models on perturbed non-relational views derived from the same source. This separation preserves answer correctness while controlling reasoning topology, table count, layout perturbations, and unit heterogeneity. Our evaluation of LLMs across financial and environmental domains reveals a critical blind spot: non-relational formatting, unit discrepancies, and multi-table scaling cause sharp, compounded performance drops.

1 Introduction

Large language models (LLMs) are increasingly used as natural-language (NL) interfaces for querying tabular data, making Table Question Answering (Table QA) a testbed for evaluating evidence retrieval, numerical operations, and analytical reasoning over structured information. However, most evaluations assume that structure is explicit, as in clean relational schemas, while many spreadsheets and reports use *non-relational* tables whose structure is encoded in the presentation itself: hierarchical headers, merged cells, blank regions, unit conventions, noisy entries, or implicit dependencies between cells (Cheng et al., 2022).

This creates a blind spot in current Table QA evaluation. What is missing is a controlled instrument that keeps the underlying evidence and answer fixed while varying how evidence is presented, perturbed, and distributed across tables. A model may answer correctly over a clean relational table, but fail when the same latent information is rendered as a non-relational view. As shown in Figure 1, and further detailed in §C, models consistently lose accuracy under non-relational renderings, with substantially larger drops in the multi-table setting. Manual construction is expensive because it requires designing realistic tables, questions, perturbations, units, missing values, and gold answers together; this limits scale, domain coverage, and diagnostic control.

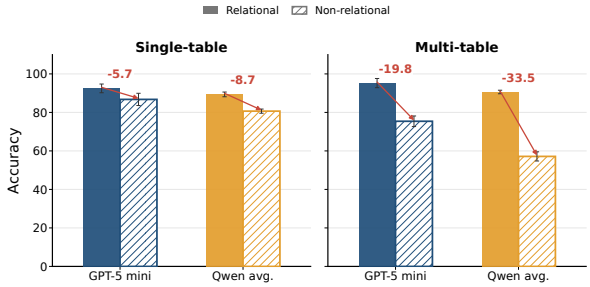


Figure 1: Motivating comparison between relational and non-relational presentations: each question evaluated on the two variants of the same underlying data in both single- and multi-table settings. Absolute Table QA performance of `gpt-5-mini` and Qwen-family on a sample of 200 questions from Spider and Kaggle datasets.

As manually curated tabular benchmarks also raise contamination concerns (Ranaldi et al., 2024), automatic benchmark generation is a natural alternative. Existing generators create templated multi-table SQL queries (Papicchio et al., 2023), use database constraints for verifiable questions (Oh et al., 2024), or synthesize SQL programs over structured table–text collections (Park et al., 2026). However, they remain tied to relational sources, with explicit schemas, predefined join paths, and questions constrained by existing database rela-

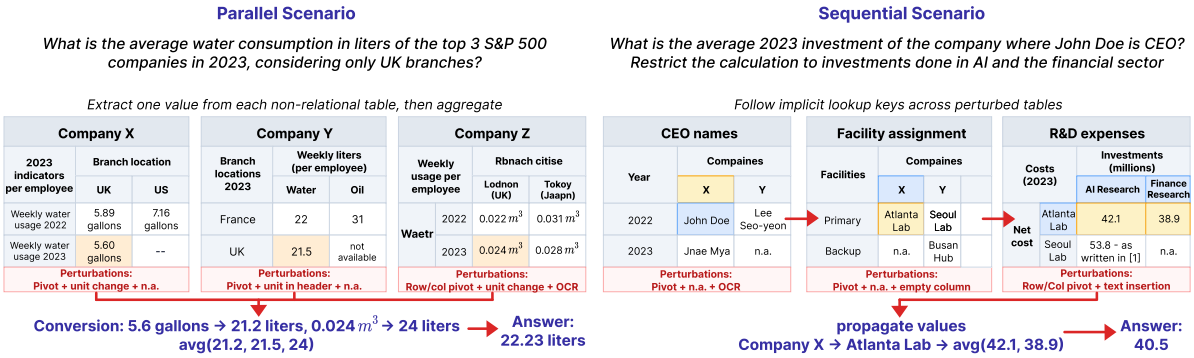


Figure 2: Multi-table parallel and sequential scenarios generated by GRADINO over non-relational tables. Parallel questions extract and aggregate values across perturbed tables; sequential questions propagate lookup keys across tables before aggregating final values.

tions. Non-relational Table QA benchmarks span domains such as finance, science, and environmental reporting (see §2), but are manually curated, fixed in domain and question distribution, and only rarely target genuinely multi-table reasoning (Zhao et al., 2022; Contalbo et al., 2025). They therefore offer limited control over table count, reasoning regime, perturbation type, and target domain.

Moreover, existing relational datasets do not easily support non-relational synthesis. As we show in §C, only a small fraction of Spider (Yu et al., 2018), BIRD (Li et al., 2023), and BEAVER (Chen et al., 2024) tables can be converted into dense pivoted or hierarchical layouts without invalid cells, and multi-table constraints are even harder to satisfy. Thus, relational datasets do not provide a flexible substrate for controlled non-relational benchmark generation. Beyond this practical obstacle, multi-table non-relational QA differs from relational joining: in *parallel* settings, models aggregate independent evidence across tables that share neither join keys nor a common schema; in *sequential* settings, they propagate intermediate entities through implicit semantic links rather than explicit keys (Figure 2). A useful benchmark must also vary layouts, surface forms, and unit conventions independently while preserving the gold answer.

To make such controlled evaluation possible, we introduce GRADINO, a flexible from-scratch framework for generating automatically verifiable QA benchmarks over multiple non-relational tables. GRADINO requires only a target domain and lightweight generation controls from the user, such as attribute cardinalities and table count. GRADINO then uses an LLM to generate a domain-specific schema, instantiates a *latent relational source*, derives executable SQL supervision and scalar answers, and verbalizes the SQL programs into NL questions.

Starting from the latent source, GRADINO generates the benchmark inputs: perturbed *non-relational views*, i.e., table representations with controlled layout, noise, and numerical variation. These views combine perturbations previously shown to challenge relational QA models (Gan et al., 2021; Pi et al., 2022; Chang et al., 2023; Zhang et al., 2025a), such as schema edits, distractors, and noisy or missing values, with transformations tailored to non-relational tables, including pivots, hierarchical headers, merged cells, layout restructuring, and unit conversions. In multi-table scenarios, GRADINO also controls how evidence is distributed across views, supporting *parallel* aggregation and *sequential* propagation. The core idea is to separate supervision from presentation: the latent relational source is used to compute and verify answers, while the perturbed non-relational views form the evaluation data shown to the model. This lets GRADINO vary reasoning topology, perturbations, and answer distributions while scaling the table count to arbitrarily high values, all without changing the gold answers.

We evaluate GRADINO on financial and environmental scenarios with up to 20 parallel tables and 5 sequentially connected tables per question, using `gpt-5-mini` and Qwen-family models. Accuracy drops by 25–40 points as the number of tables increases; in the hardest settings, performance remains slightly above 60% for parallel questions and often below 50% for sequential ones. Unit heterogeneity compounds these drops, with domain-dependent effects of up to 50 points. Although GRADINO is primarily designed for controlled evaluation, we also find that generated instances

Class	Approach	Generated artifacts			Reasoning / evaluation control			
		Tbl.	NLQ	Ans.	Pert.	Rel.	Non-rel.	# tables
Perturbation	Dr.Spider (Chang et al., 2023)	✗	✗	✗	Sch/Txt/Cell	✓	✗	Bounded
	EvoSchema (Zhang et al., 2025a)	✗	✗	✗	Sch	✓	✗	Bounded
	ADVETA (Pi et al., 2022)	✗	✗	✗	Sch	✓	✗	Bounded
	FREB-TQA (Zhou et al., 2024)	✗	✗	✗	Cell/Lay	✗	✓	1
Tabular synthesis	REaLTabFormer (Solatorio and Dupriez, 2023)	✓	✗	✗	✗	✓	✗	N/A
	GOGGLE (Liu et al., 2023)	✓	✗	✗	✗	✓	✗	N/A
	TabGen-ICL (Fang et al., 2025)	✓	✗	✗	✗	✓	✗	N/A
QA / SQL synthesis	Liu et al. (2022)	✗	✗	✓	✗	✓	✗	1
	Jiang et al. (2022)	✗	✓	✓	✗	✓	✗	1
	Pal et al. (2023)	✗	✗	✓	✗	✓	✗	Bounded
	ER-Bench (Oh et al., 2024)	✗	✓	✓	✗	✓	✗	Bounded
	SPARTA (Park et al., 2026)	✗	✓	✓	✗	✓	✗	Bounded
	DSQG-Syn (Duan et al., 2025)	✗	✓	✓	✗	✓	✗	Bounded
	SQLForge (Guo et al., 2025)	✓	✓	✓	✗	✓	✗	Bounded
	SYNQL (Baumgartner and Kornuta, 2024)	✗	✓	✓	✗	✓	✗	Bounded
Ours	**GRADINO**	✓	✓	✓	**Sch/Cell/Lay/Num**	✓	✓	**Unbounded**

Table 1: Comparison with benchmark and data-generation approaches. Tbl., NLQ, and Ans. denote generated tables, natural-language questions, and automatically verifiable answers. Perturbations are abbreviated as schema/name (Sch), text/query (Txt), cell/content (Cell), layout/structure (Lay), and numerical/unit (Num). The number of tables is either fixed to one, bounded by an existing source, or generator-controlled. GRADINO is the only approach to generate tabular benchmarks from scratch and apply perturbations without any constraint on the table number.

can provide useful supervision: fine-tuning on GRADINO-generated data improves performance on GRI-QA (Contalbo et al., 2025) compared to the non-finetuned Qwen baseline, especially when the generated data match the target domain.

In summary, our contributions are: (i) we introduce GRADINO, a from-scratch generator for automatically verifiable numerical QA over multiple non-relational tables; (ii) we define two controllable multi-table reasoning regimes, parallel and sequential, together with answer-preserving structural, cell-level, and unit-based perturbations; (iii) we use GRADINO to evaluate recent LLMs in financial and environmental domains, showing that non-relational presentation, unit heterogeneity, and table count substantially degrade performance; (iv) we publicly release code and generated datasets[1].

2 Related Work

Table QA benchmarks and robustness to tabular perturbations. Early Table QA benchmarks focused on relational tables, from single-table semantic parsing (Zhong et al., 2017) to multi-table relational QA (Yu et al., 2018; Wu et al., 2025a; Li et al., 2023; Chen et al., 2024). Subsequent studies showed that these systems remain brittle under schema, query, cell, and layout perturbations (Gan et al., 2021; Pi et al., 2022; Chang et al., 2023; Zhang et al., 2025a; Zhou et al., 2024; Liu et al., 2024), as well as in realistic or domain-specific sources (Fürst et al., 2025; Qiu et al., 2024).

Non-relational Table QA benchmarks instead target semi-structured, hierarchical, or hybrid table-text inputs, where merged headers, layout cues, and irregular cell links make direct Text-to-SQL execution unsuitable (Pasupat and Liang, 2015; Herzig et al., 2021; Chen et al., 2020a; Nan et al., 2022; Cheng et al., 2022; Wu et al., 2025b; Chen et al., 2020b, 2021a). Domain-specific datasets in finance, science, aviation, education, and environmental reporting add terminology and numerical reasoning challenges (Zhu et al., 2021; Chen et al., 2021b, 2022; Deng et al., 2022; Reddy et al., 2024; Katsis et al., 2022; Lu et al., 2023; Zhang et al., 2025b; Ghosh et al., 2024), while multi-table non-relational benchmarks require evidence aggregation across tables (Zhao et al., 2022).

These resources are valuable, but they are costly to create for new domains and offer limited control for auditing specific failure modes. GRADINO instead generates controlled non-relational Table QA instances, combining known robustness perturbations with structural and numerical transformations

[1] Anonymous Github link

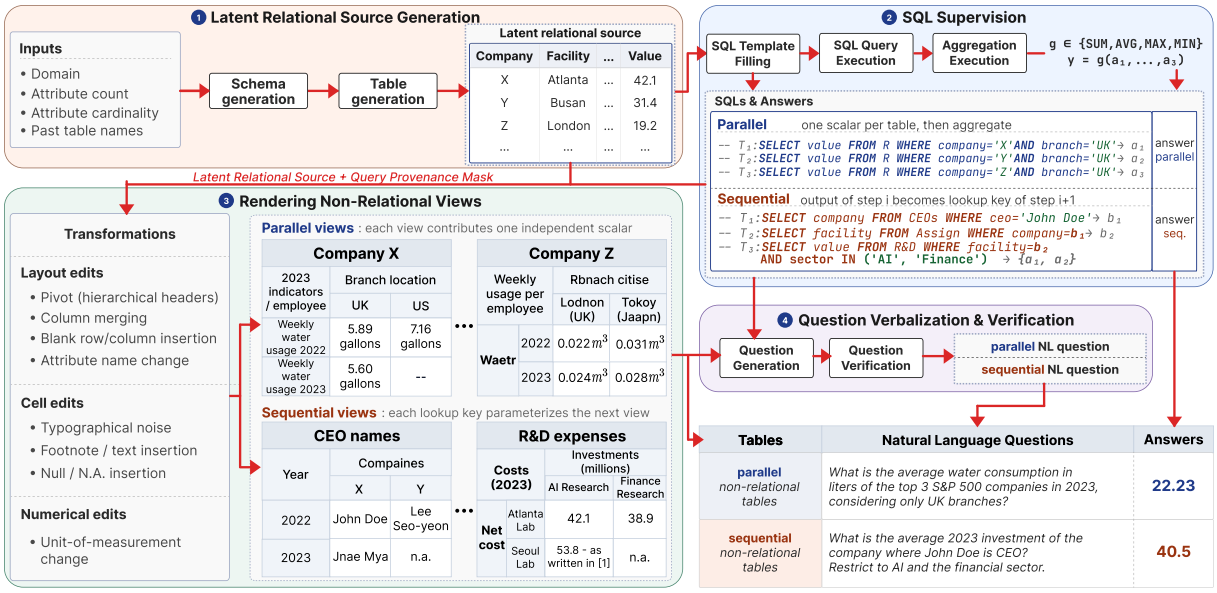


Figure 3: Overview of GRADINO. A latent relational source provides SQL supervision and gold answers, while models are evaluated only on perturbed non-relational views.

such as pivots, hierarchical headers, and unit conversions. This enables scalable stress tests over controllable domains, table counts, and multi-table reasoning settings (Contalbo et al., 2025).

Automatic generation of tabular data. Synthetic Table QA and Text-to-SQL generation reduce annotation cost by sampling executable programs and verbalizing them into natural language, using SQL templates, query-diversification methods, or relational multi-hop constructions (Liu et al., 2022; Jiang et al., 2022; Pal et al., 2023; Baumgartner and Kornuta, 2024; Guo et al., 2025; Duan et al., 2025; Oh et al., 2024; Park et al., 2026). These methods usually start from existing relational databases, so they inherit fixed schemas, table counts, and question spaces. A separate line generates synthetic tables from examples or learned relational structure (Fang et al., 2025; Solatorio and Dupriez, 2023; Liu et al., 2023; Hoppe et al., 2025), but targets tabular synthesis rather than QA.

In contrast, GRADINO generates latent sources, non-relational views, executable answers, and multi-table questions from scratch, while allowing user guidance when higher domain realism is needed (Ahmad et al., 2025) (see §B).

3 Approach

GRADINO follows the pipeline shown in Figure 3. Given a domain specification and a small set of generation controls, GRADINO constructs benchmark instances in four stages: (i) it generates a *latent relational source* (§3.1); (ii) it applies executable SQL programs and computes their gold answers (§3.2); (iii) it renders the latent source as one or more perturbed *non-relational views* (§3.3); and (iv) it verbalizes the SQL programs into NL questions (§3.4). The latent relational source is used only inside GRADINO for supervision and verification; the evaluated LLM receives only the non-relational views and the natural-language question. This separation lets us vary table layout, noise, units, and multi-table reasoning structure without changing the gold answer.

3.1 Latent Relational Source Generation

This stage constructs the latent relational source in two steps. First, GRADINO prompts an LLM to generate a domain-specific schema, conditioned on a user-defined target domain, attribute count, and attribute cardinalities, together with the names of previously generated tables, which are supplied automatically by the system from prior generation steps. This encourages related but non-identical tables. The schema specifies the table name; the attributes, each with its type and a corresponding value range; unit and decimal precision metadata; and a designated numerical attribute for quantitative reasoning. To steer generation toward realistic domains, the prompt can be grounded in example tables and domain-specific unit inventories, such as the Quantities, Units, Dimensions and Types ontology (QUDT) (qud) for environmental data and the XBRL Units Registry (XBRL UTR) (XBRL

International) for financial data.

Second, GRADINO instantiates the schema as a dense latent relational source by enumerating combinations of categorical attributes and sampling the numerical field. During instantiation, it enforces lightweight semantic regularities, including column dependencies (e.g., city $\longrightarrow$ country) and cross-row numerical constraints (e.g., a "gross" value exceeds its corresponding "net" value for the same entity and period). We defer their formalization and solver implementation to §B.

The resulting latent relational source serves as the unified basis for SQL supervision, question generation, and the construction of multiple non-relational views. Rather than generating these views independently, GRADINO derives them from the same latent representation through table-specific projections and transformations. This preserves ground-truth correspondences across views while enabling controlled reasoning topologies and automatically verifiable answers.

3.2 SQL Supervision and Multi-Table Reasoning

This stage uses SQL as internal supervision: GRADINO executes SQL queries over the latent relational source to compute gold labels; the evaluated LLM receives only the non-relational views and the NL question. We focus on aggregation-style questions that combine scalar values.

In the **parallel** setting, GRADINO constructs a predicate over categorical attributes that isolates a target numerical value. It instantiates multiple table-specific selections by keeping the predicate fixed except for one attribute that varies across views (e.g., keeping the sector fixed while changing the company; see Figure 3). Each view contributes one scalar a_i, and the final answer is computed as $y = g(a_1, \ldots, a_n)$, with $g \in \{\mathrm{sum}, \mathrm{avg}, \mathrm{max}, \mathrm{min}\}$. Thus, parallel instances correspond to multi-table sum, average, and superlative questions, with one extractive SQL query per view and one aggregation over the scalars.

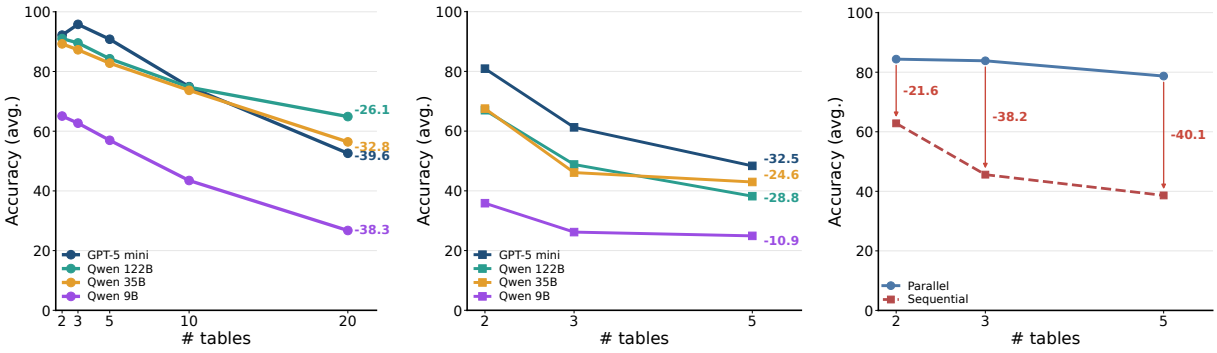
In the **sequential** setting, GRADINO constructs a chain of table-local queries. Intermediate views are associated with extractive queries whose returned value becomes the only lookup key for the next view in the chain, while the final view applies the target numerical aggregation. The final aggregation uses the same operators as in the parallel setting, applied to a randomly sampled set of 2 to 5 values returned by the last view. The gold label is the result of executing this latent composition. This setup yields multi-step reasoning across tables without relying on an externally given relational schema. For instance, the example in Figure 3 requires identifying the company associated with John Doe, using that company to retrieve the AI and financial-sector investment values, and averaging them.

Since the same latent source can be reused to generate multiple types of questions and different numbers of views, the approach scales naturally between reasoning depths and evidence cardinalities.

3.3 Rendering Non-Relational Views

Rendering is organized as an answer-preserving transformation sequence. GRADINO first creates table-specific relational projections from the latent source and records a provenance mask marking cells required by the SQL program and cells safe to perturb. It then introduces null values, and applies column merging, unit rendering and conversion while the representation is still relational. Next, GRADINO converts each projection into a non-relational view through a pivot-style transformation, organizing selected attributes as hierarchical row and column headers. The generator searches over candidate row/column partitions, prefers dense pivots, shrinks the layout, and retains all evidence required by the SQL program. In all experiments, rendered views are capped at 10×10 cells. After layout construction, GRADINO applies local perturbations such as blank rows and columns, typographical noise, and contextual or footnote-like text.

The same rendering machinery supports both multi-table regimes, with scenario-specific controls. In parallel generation, each view provides one independently retrievable scalar for the final aggregation, and unit conversion can force normalization across compatible quantities. In sequential generation, each view represents one step of a latent lookup chain. Intermediate views contain exactly one occurrence of the lookup key extracted from the previous view. This is an evaluation choice rather than a framework limitation: ambiguous or tree-shaped joins would add uncontrolled variation in join complexity and the number of values entering the final aggregation. Unique matches keep the reasoning depth comparable across instances while still requiring models to recover implicit cross-table links, since latent joins are not stated in the question. Finally, view order is randomized to prevent shortcutting. Overall, render-



(a) Parallel questions (b) Sequential questions (c) Parallel vs sequential

Figure 4: Scaling behavior as the number of tables increases. Labels at the end of curves in (a) and (b) report the accuracy drop from the smallest to largest table count. Panel (c) compares parallel and sequential questions at matched table counts, showing that sequential reasoning adds degradation beyond multi-table aggregation.

ing produces non-relational views that hide clean schemas, explicit joins, and standardized numerical expressions but remain verifiable through the latent relational source and SQL supervision.

3.4 Question Verbalization and Verification

In the final stage, GRADINO prompts an LLM to generate a natural-language question. The prompt receives all non-relational views in HTML form, the SQL supervision, the aggregation description, and the gold answer. The multi-table verbalization differs slightly across the two reasoning regimes. For parallel questions, the generator verbalizes the shared selection pattern together with the cross-table aggregation to be performed. For sequential questions, the generator verbalizes only the first and last steps of the chain and leaves intermediate dependencies implicit in the table contents. As a result, the final question does not expose the latent composition directly, but still requires the evaluated LLM to recover it from the views. To filter out noisy generations, we perform an LLM-based verification pass after question generation. The verifier receives the generated question, the views, the latent SQL program, and the gold answer, and repairs the question whenever the verbalization is inconsistent with the underlying supervision.

4 Experimental evaluation

Our experiments are designed to answer four questions. (**RQ1**) *Scaling*: how does performance evolve as the number of tables grows, in both parallel and sequential settings? (**RQ2**) *Numerical heterogeneity*: to what extent do unit discrepancies and quantitative computation degrade accuracy, and is this effect domain-dependent? (**RQ3**) *Utility*: beyond evaluation, can GRADINO-generated data serve as effective training supervision? (**RQ4**) *Question correctness*: what is GRADINO's percentage of correctly generated questions? We address these through controlled benchmarks in the financial and environmental domains, and validate our findings on perturbed real tables in §C.

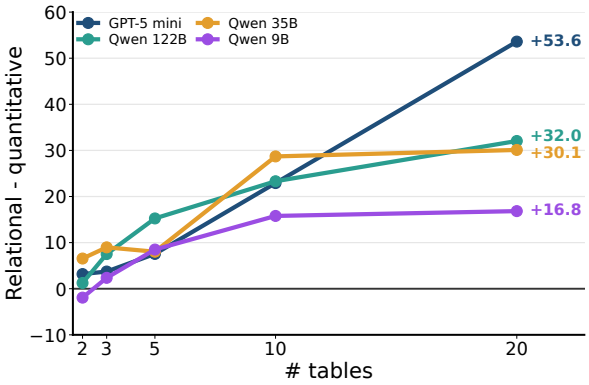


Figure 5: Performance gap between relational and quantitative parallel questions as table count increases.

4.1 Baselines

We evaluate four LLMs with different scales and access settings: `gpt-5-mini`, `Qwen3.5-122B-A10B-FP8` (Qwen 122B), `Qwen3.5-35B-A3B-FP8` (Qwen 35B), and `Qwen3.5-9B` (Qwen 9B). This covers one strong closed model and open-weight models of different sizes, testing whether failures exposed by GRADINO are model-family specific or persist across scale. All models use zero-shot Chain-of-Thought (Kojima et al., 2022), with *thinking* enabled, temperature equal to 0 and a token budget of 16384 for Qwen; details are in §A. Additional experiments with Program-of-Thoughts prompting (Chen et al., 2023) are detailed in §D.

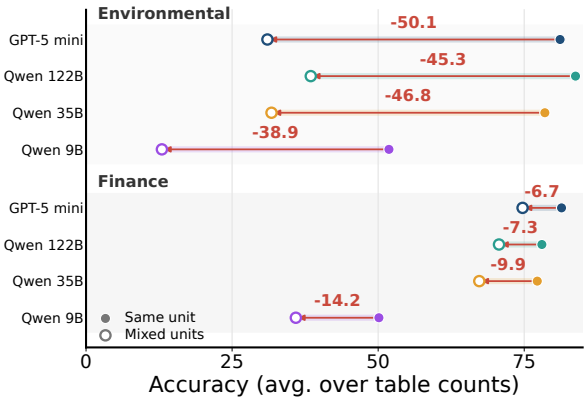


Figure 6: Effect of unit heterogeneity on parallel questions. Red arrows indicate the drop for each model and domain, averaged across different table counts.

4.2 Metrics

We report accuracy using normalized exact match (EM) against the automatically computed gold answer, after removing superficial formatting differences such as surrounding whitespace, thousands separators, and minor numerical variants. Accuracy is the percentage of examples whose normalized prediction matches the gold answer.

4.3 Datasets

We generate GRADINO instances in two domains, environmental and finance, using `gpt-5-mini` as its internal LLM. Unless otherwise stated, evaluation covers parallel and sequential questions over 2, 3, and 5 tables, with parallel questions also scaled to 10 and 20 tables. We request 50 instances for each combination of domain, table count, aggregation operator (sum, average, superlative), and, for parallel questions, same- or mixed-unit setting. Perturbations are applied randomly but compositionally: each larger benchmark extends the next smaller one, so 3-table instances reuse 2-table inputs, and so on. Likewise, every ablation reuses the same inputs, varying only the studied property. After discarding generations that fail the schema-format check, we obtain 2,322 parallel and 840 sequential instances; further statistics are in §E.

4.4 Discussion

(**RQ1**) Scaling collapses performance. As shown in Figure 4, accuracy decreases as the number of tables increases in both parallel and sequential settings. In parallel scenarios, scaling from 2 to 20 tables causes substantial drops across all models (Figure 4a): `gpt-5-mini` decreases by 39.6 points, while Qwen 122B, Qwen 35B, and Qwen 9B lose 26.1, 32.8, and 38.3 points, respectively. Although `gpt-5-mini` achieves the highest performance with few tables (95.53% at 3 tables), accuracy collapses at larger scales, reaching only 52.61% at 20 tables. At the same point, Qwen 122B, Qwen 35B, and Qwen 9B obtain 64.90%, 56.42%, and 26.74%, respectively.

Sequential settings show the same pattern (Figure 4b): extending the lookup chain from 2 to 5 tables reduces accuracy by 32.5 points for `gpt-5-mini`, 28.8 for Qwen 122B, 24.6 for Qwen 35B, and 10.9 for Qwen 9B. At 5 tables, performance ranges from 48.37% for `gpt-5-mini` to 24.93% for Qwen 9B.

Comparing the two regimes (Figure 4c), sequential reasoning is consistently harder than parallel aggregation at matched table counts. The average gap grows from 21.6 points with 2 tables to 40.1 points with 5 tables, showing that propagating lookup keys across tables adds difficulty beyond aggregating independent evidence. However, the two regimes stress models differently: sequential questions are harder at the same table count, while parallel questions can scale to many more tables, where the scaling collapse makes their difficulty comparable to shorter sequential chains.

(**RQ2**) Quantitative reasoning and unit heterogeneity are a major bottleneck. The observed scaling collapse is largely driven by quantitative questions. In Figure 5, we report the average gap between relational and quantitative parallel questions. All models exhibit a consistent trend: the gap increases as the number of tables grows, indicating that numerical computation becomes progressively harder when more table-specific values must be extracted and aggregated. At 20 tables, `gpt-5-mini` reaches the largest degradation, with quantitative questions underperforming relational ones by 53.6 points. The same pattern holds for Qwen 122B, Qwen 35B, and Qwen 9B, with gaps of 32.0, 30.1, and 16.8 points, respectively. Overall, quantitative reasoning accounts for an increasing share of errors as table complexity grows.

This effect is further amplified by unit heterogeneity. As shown in Figure 6, adding heterogeneous units on top of the same parallel settings leads to additional performance drops, with a strong domain dependence. The effect is particularly pronounced in the environmental domain,

Train data	2		3		5		
	rel	qnt	rel	qnt	rel	qnt	avg
None	18.3	9.3	17.1	9.7	9.2	0.0	10.6
GRADINO (fin)	22.7	47.8	21.6	19.6	10.8	7.5	$21.7^{+11.1}$
GRADINO (env)	39.0	45.7	34.5	17.7	23.3	7.5	$27.9^{+17.3}$
TAT-QA	43.6	43.5	28.8	13.7	10.0	2.5	$23.7^{+13.1}$
GRI-QA	70.9	47.8	51.1	11.8	30.8	2.5	$35.8^{+25.2}$

Table 2: Accuracy of Qwen2.5-14B-Instruct fine-tuned, evaluated on the GRI-QA test split, by number of input tables (2/3/5) and question type (relational, quantitative). Superscripts report the average gain over the non-finetuned baseline (None).

Topology	*Avg*	*Sum*	*Superlative*	**Mean**
Parallel	100.0	100.0	100.0	100.0
Sequential	90.0	80.0	90.0	86.7
Mean	95.0	90.0	95.0	93.3

Table 3: Manually verified correctness (%) on the hardest GRADINO-generated environmental instances, by reasoning topology and aggregation operator.

where performance decreases by 50.1 points for `gpt-5-mini`, and by 45.3, 46.8, and 38.9 points for Qwen 122B, Qwen 35B, and Qwen 9B, respectively. In contrast, the same perturbation is considerably less severe in finance, with drops of 6.7, 7.3, 9.9, and 14.2 points. This suggests that unit heterogeneity is not mere formatting, but a domain-dependent reasoning challenge. Environmental questions often involve heterogeneous physical conversions (e.g., gallons to liters), whereas finance mostly uses scale conversions (e.g., thousands to billions). Thus, robustness to numerical representations may not transfer across domains: domain-specific table conventions can induce distinct failure modes that affect model performance (Santacroce et al., 2026).

(**RQ3**) Data generated by GRADINO has functional utility. To evaluate the utility, we fine-tune `Qwen2.5-14B-Instruct` using correct `gpt-5-mini` reasoning traces derived from GRADINO instances. As an evaluation benchmark, we use GRI-QA (Contalbo et al., 2025), which consists of relational and quantitative questions over 2-, 3-, and 5-table environmental settings. We compare several training regimes: no finetuning, GRADINO-generated 2-, 3- and 5-table training data, and real data (financial single-table TAT-QA Zhu et al., 2021 and the GRI-QA train split). For all finetuning settings, we restrict the training data to 440 randomly sampled instances. Results are reported in Table 2. All regimes outperform the non-finetuned baseline (+11.1 to +25.2 on average), with gains strongly depending on domain alignment: environmental data consistently outperforms financial data due to better match with the GRI-QA test domain. Notably, GRADINO exposes models to multi-table and quantitative settings largely absent in real single-table benchmarks: while TAT-QA fine-tuning is competitive at 2 tables (43.6 on average over relational and quantitative questions), it degrades sharply with scale (6.3 at 5 tables). In contrast, GRADINO (environmental) shows a more stable trend (15.4 at 5 tables).

Finally, the GRI-QA train split, being in-domain, yields the strongest results, yet its gains concentrate on relational rather than quantitative questions, likely reflecting the scarcity of quantitative examples in its training split. This underscores that the scalable, quantitative-rich generation of GRADINO is needed to improve numerical reasoning at scale.

(**RQ4**) Question generation is reliable. Question generation is the only GRADINO stage that can introduce errors. To estimate its reliability, we manually inspected 10 instances per aggregation operator (sum, average, superlative) in the environmental domain, covering parallel (20 tables, unit-converted) and sequential (5 tables) regimes, for 60 total. As shown in Table 3, all parallel instances are correct, while sequential ones average 86.7%. The gap reflects higher verbalization complexity in sequential settings, which must coordinate cross-table constraints and are more prone to missing `WHERE` clauses. While GRADINO does not guarantee question correctness by construction, its measured reliability exceeds that reported by other real tabular benchmarks (Park et al., 2026).

5 Conclusion

We introduced GRADINO, a framework for generating automatically verifiable numerical QA benchmarks over multiple non-relational tables. Our experiments show that recent LLMs remain brittle in this setting, and suffering large domain-dependent failures under heterogeneous units. Overall, we identify multi-table non-relational numerical reasoning as a persistent blind spot for current LLMs and provide a controlled benchmark-generation framework for studying it.

6 Limitations

Synthetic data realism. Our experiments on perturbed real tables validate the observed degradation on generated tables. However, GRADINO is designed as a controlled diagnostic generator, not as a replacement for manually curated benchmarks built from naturally occurring documents. Although the generator is conditioned on domain descriptions, unit inventories, and optional examples, the resulting tables may not capture all distributional, visual, and semantic regularities of real spreadsheets or reports, not even with GRADINO applied in a semi-automatic manner (§B.4). This is a deliberate trade-off: from-scratch generation gives us control over table count, reasoning topology, perturbations, and units, but it does not guarantee full realism.

Restricted numerical structure. This study focuses on numerical QA, where each latent table contains a distinguished numerical value attribute. Although GRADINO can also generate non-numerical questions, we restrict our analysis to numerical questions because parallel scenarios, which require evidence retrieval from multiple tables followed by an aggregation operation, are especially common with numerical values. Future work could extend GRADINO to non-numerical settings, including questions that require set operations, categorical comparisons, or logical aggregation over textual evidence.

Question verbalization. Gold answers are computed by executing latent SQL programs, and non-relational views are generated through answer-preserving transformations. However, the NL question is produced by an LLM and is therefore not guaranteed correct by construction. We mitigate this issue through verifier-based repair and manual inspection. Table 3 shows high reliability overall, but also that sequential questions are more error-prone, mainly because verbalization may drop latent selection constraints. Thus, GRADINO provides executable guarantees for answers and evidence preservation, while question fidelity remains only an empirically validated component.

Generation failures and filtering. Some generations fail schema-format checks. We discard invalid generations, which improves benchmark quality but increases the cost of data generation. Future work should focus on reducing the number of discarded generations to maximize QA generation throughput.

Perturbation coverage. We consider a broad set of structural, cell-level, layout, and numerical perturbations. Nevertheless, real non-relational tables contain additional phenomena that we do not fully model, such as visual alignment cues and chart-table mixtures. Although GRADINO could be extended to cover those perturbations, the current benchmark does not stress the full range of document-table understanding. We leave this as future work.

7 Ethical considerations

Data and privacy. GRADINO generates its evaluation data entirely from scratch: latent relational sources, tables, and questions are synthesized by LLMs from a domain specification and contain only fictional entities and randomly sampled numerical values. The generated instances therefore include no personally identifying information and no real individuals, and we collect no scraped or user-contributed content. The real tables used in our validation experiments (§C) come from public databases used exclusively for evaluation (Northwind, BikeStores, SQL Bootcamp) and from existing research benchmarks (Spider, BIRD, BEAVER, TAT-QA, GRI-QA) built from public corporate and web sources. Where such sources mention named public figures, this information is already public and is neither augmented nor inferred by GRADINO.

Use of existing artifacts. We use all datasets and models in a manner consistent with their intended research use, and we report the license of each artifact in §G. Our use is non-commercial and research-only, and any derivatives we produce remain within a research context. The crawled sample databases are used solely for testing, without redistribution (§C).

Risks and broader impact. GRADINO is a diagnostic tool for measuring the robustness of Table QA systems, and we see limited potential for direct misuse. The principal risk is interpretive: because the benchmark is synthetic and controlled, strong performance on GRADINO should not be read as a guarantee of real-world reliability, nor should low scores be taken as evidence that a model is unusable in practice. We therefore position GRADINO as a complement to, rather than a replacement for, manually curated benchmarks built from naturally occurring documents (§6). Used appropriately,

surfacing the multi-table, non-relational, and unit-heterogeneity failure modes, we believe GRADINO can support the development of more reliable analytical interfaces over tabular data, including in financial and environmental reporting settings where silent numerical errors carry real consequences.

Human verification and AI assistants. The correctness study in Table 3 was carried out by the authors and involves no external annotators or human subjects, raising no participant-compensation or consent concerns. Our use of AI assistants for writing is described in §8, and the role of LLMs within the generation pipeline is described in §3.

8 Use of AI assistants

When writing this paper, we used ChatGPT to improve the flow of writing and the vocabulary of the initial drafts we manually wrote. Each suggestion has been manually validated by the authors.

References

QUDT: Quantities, units, dimensions and types ontologies. https://www.qudt.org/. Accessed: 2026-05-09.

Mohammad Shahmeer Ahmad, Zan Ahmad Naeem, Michaël Aupetit, Ahmed K. Elmagarmid, Mohamed Y. Eltabakh, Xiaosong Ma, Mourad Ouzzani, and Chaoyi Ruan. 2025. HCT-QA: A benchmark for question answering on human-centric tables. *CoRR*, abs/2504.20047.

Denver Baumgartner and Tomasz Kornuta. 2024. SynQL: Synthetic data generation for in-domain, low-resource text-to-SQL parsing. In *NeurIPS 2024 Third Table Representation Learning Workshop*.

Shuaichen Chang, Jun Wang, Mingwen Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Joseph Lilien, Steve Ash, William Yang Wang, Zhiguo Wang, Vittorio Castelli, Patrick Ng, and Bing Xiang. 2023. Dr.spider: A diagnostic evaluation benchmark towards text-to-SQL robustness. In *The Eleventh International Conference on Learning Representations*.

Peter Baile Chen, Fabian Wenz, Yi Zhang, Moe Kayali, Nesime Tatbul, Michael J. Cafarella, Çagatay Demiralp, and Michael Stonebraker. 2024. BEAVER: an enterprise benchmark for text-to-sql. *CoRR*, abs/2409.02038.

Wenhu Chen, Ming-Wei Chang, Eva Schlinger, William Yang Wang, and William W. Cohen. 2021a. Open question answering over tables and text. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*.

Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. 2020a. Tabfact: A large-scale dataset for table-based fact verification. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net.

Wenhu Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. 2020b. Hybridqa: A dataset of multi-hop question answering over tabular and textual data. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, Findings of ACL, pages 1026–1036. Association for Computational Linguistics.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Kenneth Huang, Bryan R. Routledge, and William Yang Wang. 2021b. Finqa: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pages 3697–3711. Association for Computational Linguistics.

Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. 2022. Convfinqa: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pages 6279–6292. Association for Computational Linguistics.

Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. 2022. Hitab: A hierarchical table dataset for question answering and natural language generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, pages 1094–1110. Association for Computational Linguistics.

Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering denial constraints. *Proc. VLDB Endow.*, 6(13):1498–1509.

Michele Luca Contalbo, Sara Pederzoli, Francesco Del Buono, Venturelli Valeria, Francesco Guerra, and Matteo Paganelli. 2025. GRI-QA: a comprehensive benchmark for table question answering over environmental data. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 15764–15779, Vienna, Austria. Association for Computational Linguistics.

Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: an efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS'08/ETAPS'08, page 337–340, Berlin, Heidelberg. Springer-Verlag.

Yang Deng, Wenqiang Lei, Wenxuan Zhang, Wai Lam, and Tat-Seng Chua. 2022. PACIFIC: towards proactive conversational question answering over tabular and textual data in finance. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pages 6970–6984. Association for Computational Linguistics.

Shaoming Duan, Youxuan Wu, Chuanyi Liu, Yuhao Zhang, Zirui Wang, Peiyi Han, Shengyuan Yu, Liang Yan, and Yingwei Liang. 2025. DSQG-syn: Synthesizing high-quality data for text-to-SQL parsing by domain specific question generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2971–2989, Albuquerque, New Mexico. Association for Computational Linguistics.

Liancheng Fang, Aiwei Liu, Hengrui Zhang, Henry Peng Zou, Weizhi Zhang, and Philip S. Yu. 2025. TABGEN-ICL: Residual-aware in-context example selection for tabular data generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20027–20041, Vienna, Austria. Association for Computational Linguistics.

Jonathan Fürst, Catherine Kosten, Farhad Nooralahzadeh, Yi Zhang, and Kurt Stockinger. 2025. Evaluating the data model robustness of text-to-sql systems based on real user queries. In *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, pages 158–170. OpenProceedings.org.

Yujian Gan, Xinyun Chen, Qiuping Huang, Matthew Purver, John R. Woodward, Jinxia Xie, and Pengsheng Huang. 2021. Towards robustness of text-to-SQL models against synonym substitution. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2505–2515, Online. Association for Computational Linguistics.

Akash Ghosh, Venkata Sahith Bathini, Niloy Ganguly, Pawan Goyal, and Mayank Singh. 2024. How robust are the QA models for hybrid scientific tabular data? a study using customized dataset. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 8258–8264, Torino, Italia. ELRA and ICCL.

Yu Guo, Dong Jin, Shenghao Ye, Shuangwu Chen, Jian Yang, and Xiaobin Tan. 2025. SQLForge: Synthesizing reliable and diverse data to enhance text-to-SQL reasoning in LLMs. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 8441–8452, Vienna, Austria. Association for Computational Linguistics.

Jonathan Herzig, Thomas Müller, Syrine Krichene, and Julian Martin Eisenschlos. 2021. Open domain question answering over tables via dense retrieval. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*, pages 512–519. Association for Computational Linguistics.

Frederik Hoppe, Astrid Franz, Lars Kleinemeier, and Udo Göbel. 2025. Generating synthetic relational tabular data via structural causal models. In *Proceedings of the 4th Table Representation Learning Workshop*, pages 13–18, Vienna, Austria. Association for Computational Linguistics.

Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Zhengbao Jiang, Yi Mao, Pengcheng He, Graham Neubig, and Weizhu Chen. 2022. OmniTab: Pretraining with natural and synthetic data for few-shot table-based question answering. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 932–942, Seattle, United States. Association for Computational Linguistics.

Yannis Katsis, Saneem Chemmengath, Vishwajeet Kumar, Samarth Bharadwaj, Mustafa Canim, Michael Glass, Alfio Gliozzo, Feifei Pan, Jaydeep Sen, Karthik Sankaranarayanan, and Soumen Chakrabarti. 2022. AIT-QA: Question answering dataset over complex tables in the airline industry. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Track*, pages 305–314, Hybrid: Seattle, Washington + Online. Association for Computational Linguistics.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Jinyang Li, Binyuan Hui, GE QU, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. 2022. TAPEX: table pre-training via learning a neural SQL executor. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.

Tennison Liu, Zhaozhi Qian, Jeroen Berrevoets, and Mihaela van der Schaar. 2023. GOGGLE: Generative modelling for tabular data by learning relational structure. In *The Eleventh International Conference on Learning Representations*.

Tianyang Liu, Fei Wang, and Muhao Chen. 2024. Rethinking tabular data understanding with large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 450–482, Mexico City, Mexico. Association for Computational Linguistics.

Pan Lu, Liang Qiu, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, Tanmay Rajpurohit, Peter Clark, and Ashwin Kalyan. 2023. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. In *The Eleventh International Conference on Learning Representations*.

Linyong Nan, Chiachun Hsieh, Ziming Mao, Xi Victoria Lin, Neha Verma, Rui Zhang, Wojciech Kryscinski, Hailey Schoelkopf, Riley Kong, Xiangru Tang, Mutethia Mutuma, Ben Rosand, Isabel Trindade, Renusree Bandaru, Jacob Cunningham, Caiming Xiong, and Dragomir R. Radev. 2022. Fetaqa: Free-form table question answering. *Trans. Assoc. Comput. Linguistics*, 10:35–49.

Jio Oh, Soyeon Kim, Junseok Seo, Jindong Wang, Ruochen Xu, Xing Xie, and Steven Whang. 2024. Erbench: An entity-relationship based automatically verifiable hallucination benchmark for large language models. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.

Vaishali Pal, Andrew Yates, Evangelos Kanoulas, and Maarten de Rijke. 2023. MultiTabQA: Generating tabular answers for multi-table question answering. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6322–6334, Toronto, Canada. Association for Computational Linguistics.

Simone Papicchio, Paolo Papotti, and Luca Cagliero. 2023. QATCH: benchmarking sql-centric tasks with table representation learning models on your data. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Sungho Park, Jueun Kim, and Wook-Shin Han. 2026. SPARTA: Scalable and principled benchmark of tree-structured multi-hop QA over text and tables. In *The Fourteenth International Conference on Learning Representations*.

Panupong Pasupat and Percy Liang. 2015. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing, ACL 2015, July 26-31, 2015, Beijing, China, Volume 1: Long Papers*, pages 1470–1480. The Association for Computer Linguistics.

Xinyu Pi, Bing Wang, Yan Gao, Jiaqi Guo, Zhoujun Li, and Jian-Guang Lou. 2022. Towards robustness of text-to-SQL models against natural and realistic adversarial table perturbation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2007–2022, Dublin, Ireland. Association for Computational Linguistics.

Zipeng Qiu, You Peng, Guangxin He, Binhang Yuan, and Chen Wang. 2024. Tqa-bench: Evaluating llms for multi-table question answering with scalable context and symbolic extension. *CoRR*, abs/2411.19504.

Joeri Rammelaere and Floris Geerts. 2019. Revisiting conditional functional dependency discovery: Splitting the "c" from the "fd". In *Machine Learning and Knowledge Discovery in Databases*, pages 552–568, Cham. Springer International Publishing.

Federico Ranaldi, Elena Sofia Ruzzetti, Dario Onorati, Leonardo Ranaldi, Cristina Giannone, Andrea Favalli, Raniero Romagnoli, and Fabio Massimo Zanzotto. 2024. Investigating the impact of data contamination of large language models in text-to-SQL translation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13909–13920, Bangkok, Thailand. Association for Computational Linguistics.

Varshini Reddy, Rik Koncel-Kedziorski, Viet Dac Lai, Michael Krumdick, Charles Lovering, and Chris Tanner. 2024. DocFinQA: A long-context financial reasoning dataset. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 445–458, Bangkok, Thailand. Association for Computational Linguistics.

Marta Santacroce, Michele Luca Contalbo, Sara Pederzoli, Riccardo Benassi, Venturelli Valeria, Matteo Paganelli, and Francesco Guerra. 2026. CLARIESG: An end-to-end system for ESG analysis over complex tables in corporate reports. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 86–100, Rabat, Marocco. Association for Computational Linguistics.

Aivin V. Solatorio and Olivier Dupriez. 2023. Realtabformer: Generating realistic relational and tabular data using transformers. *CoRR*, abs/2302.02041.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508, Toronto, Canada. Association for Computational Linguistics.

Jian Wu, Linyi Yang, Dongyuan Li, Yuliang Ji, Manabu Okumura, and Yue Zhang. 2025a. MMQA: Evaluating LLMs with multi-table multi-hop complex questions. In *The Thirteenth International Conference on Learning Representations*.

Pengzuo Wu, Yuhang Yang, Guangcheng Zhu, Chao Ye, Hong Gu, Xu Lu, Ruixuan Xiao, Bowen Bao, Yijing He, Liangyu Zha, Wentao Ye, Junbo Zhao, and Haobo Wang. 2025b. Realhitbench: A comprehensive realistic hierarchical table benchmark for evaluating llm-based table analysis. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Findings of ACL, pages 7105–7137. Association for Computational Linguistics.

XBRL International. XBRL units registry 1.0. `https://specifications.xbrl.org/work-product-index-registries-units-registry-1.0.html`. Accessed: 2026-05-09.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, Brussels, Belgium. Association for Computational Linguistics.

Tianshu Zhang, Kun Qian, Siddhartha Sahai, Yuan Tian, Shaddy Garg, Huan Sun, and Yunyao Li. 2025a. Evoschema: Towards text-to-sql robustness against schema evolution. *Proc. VLDB Endow.*, 18(10):3655–3668.

Xuanliang Zhang, Dingzirui Wang, Baoxin Wang, Longxu Dou, Xinyuan Lu, Keyan Xu, Dayong Wu, and Qingfu Zhu. 2025b. SCITAT: A question answering benchmark for scientific tables and text covering diverse reasoning types. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 3859–3881, Vienna, Austria. Association for Computational Linguistics.

Yilun Zhao, Yunxiang Li, Chenying Li, and Rui Zhang. 2022. MultiHiertt: Numerical reasoning over multi hierarchical tabular and textual data. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6588–6600, Dublin, Ireland. Association for Computational Linguistics.

Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103.

Wei Zhou, Mohsen Mesgar, Heike Adel, and Annemarie Friedrich. 2024. FREB-TQA: A fine-grained robustness evaluation benchmark for table question answering. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, pages 2479–2497. Association for Computational Linguistics.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*, pages 3277–3287. Association for Computational Linguistics.

A Appendix Overview

This appendix provides additional methodological details and extended results complementing the main paper.

Appendix B details the lightweight semantic constraints referenced in §3.1, which GRADINO can optionally enforce while instantiating the latent relational source to push it toward more plausible configurations.

Appendix C reports the main tests cited in §1 that motivate GRADINO. In particular, it shows that perturbed parallel and sequential questions are difficult to synthesize from existing relational tables, and that non-relationalization markedly degrades LLM performance in multi-table settings.

Appendix D reports the detailed experimental results underlying §4.4, together with the finetuning configuration. It provides per-setting performance of all evaluated baselines on parallel and sequential questions over both relational and non-relational tables, as well as the training setup and data composition used in the finetuning experiments.

Appendix E reports statistics on the GRADINO-generated data used throughout the experiments, including the number of retained instances per experiment, the share of candidates discarded during generation, and how instances are balanced across table counts, aggregation operators, unit settings, and relational versus non-relational views.

Appendix F illustrates the prompts used at each stage of GRADINO, covering latent relational

source generation, semantic constraint generation, natural-language question generation and verification for both parallel and sequential regimes, and the inference prompt given to the evaluated models.

Appendix G details the licenses of all artifacts used in this work, all of which permit research use.

B Enforcing semantic constraints

This appendix describes the lightweight semantic regularities referenced in §3.1, which GRADINO can optionally enforce while instantiating the latent relational source. The goal is to push the densely sampled source toward more plausible configurations. These constraints are not part of the supervision signal and are not evaluated quantitatively; gold answers are always computed by executing the SQL supervision over the instantiated tables. The process has three steps: an LLM *defines* constraints over the schema (§B.1), the constraints are *parsed* into a solver representation (§B.2), and feasible values are *sampled* subject to them (§B.3). §B.4 further demonstrates a usage mode in which the user semi-automatically defines and validates the schema and constraints prior to QA generation with GRADINO. In this setting, constraint plausibility is determined by the user rather than left to the LLM.

B.1 Rule Definition

We adopt two constraint families. **Intra-row constraints** (Conditional Functional Dependencies Rammelaere and Geerts, 2019) bind semantically related values within a tuple: whenever a row matches a selector, one of its attributes must satisfy an (in)equality (e.g., "*if the city is Lisbon, the country must be Portugal*"). **Inter-row constraints** (Denial Constraints Chu et al., 2013) express order or arithmetic relations on the numerical value column v across rows matched on a selector (e.g., "*net income must always be lower than the gross income*"). Given the schema metadata from §3.1, an LLM is prompted to infer realistic rules for both constraint families using its internal domain knowledge. Rules use a restricted, Python-evaluable syntax: a row is selected by fixing categorical attributes, e.g. `(TypeOfMoney == "Gross")`, with all unspecified attributes implicitly held equal across matched rows. Intra-row rules take the conditional form `if (selector) then (predicate)`, e.g. `if (CompanyTier == "Small") then (Amount <= 5000000)`, while inter-row rules compare the value column of two selected rows through the attribute-as-property syntax `(selector).`v, e.g. `(TypeOfMoney == "Gross").Amount > (TypeOfMoney == "Net").Amount`. More information on the rules format and allowed operations can be found inside the prompt shown in Figure 10.

B.2 Parsing into a Solver Representation

Each rule is translated into a constraint over the Z3 SMT solver (De Moura and Bjørner, 2008), using one solver variable per (attribute, row) pair. Categorical attributes are encoded as enumerated sorts over their candidate values and numerical attributes as integer or real variables. Intra-row rules become implications from the selector to the predicate, applied to the rows the selector matches; inter-row rules are grounded by substituting the value variables of the two matched rows into the comparison. The declared range of v, defined during schema generation, is added as an explicit bound.

B.3 Randomized Value Sampling

Enforcing the constraints must not collapse the table to a single canonical solution, so we sample values within their feasible region rather than returning an arbitrary satisfying model. For intra-row constraints, for each variable we use Z3 to compute its feasible interval $[\min, \max]$ given all constraints and previously fixed variables. Then, we draw a value uniformly from that interval, assign it to the variable, and then add the novel constraint inside the optimizer's constraints. For inter-row constraints, since each rule may contain multiple numerical variables belonging to different rows, optimizing all the variables may become computationally expensive. For this reason, for each variable we instead obtain two feasible values with satisfiability queries and treat them as $[\min, \max]$ values.

Because each variable is bounded conditionally on earlier choices, the final assignment is jointly consistent while remaining randomized. When the solution space is not continuous (e.g., constraints involving modulo), we do not apply randomized sampling. Rows whose constraints are unsatisfiable are dropped, and the sampled values are written back rounded to the precision declared declared during schema generation.

B.4 Enforcing constraints semi-automatically

The constraints described in §B.1–§B.3 are inferred automatically by an LLM and are therefore best-

			2			3			5			10			20		
			rel	quant	avg	rel	quant	avg	rel	quant	avg	rel	quant	avg	rel	quant	avg
environmental	same unit	gpt-5-mini	94.74	89.47	92.11	92.11	93.43	92.77	95.45	89.19	92.32	87.10	60.75	73.93	87.88	21.21	54.55
		Qwen3.5-122B	89.47	96.06	92.76	97.37	89.47	93.42	86.47	85.14	85.80	80.56	70.84	75.70	84.85	57.58	71.22
		Qwen3.5-35B	86.84	86.84	86.84	84.21	93.43	88.82	89.19	82.44	85.81	88.89	58.34	73.61	78.79	36.36	57.58
		Qwen3.5-9B	65.79	67.11	66.45	65.79	67.11	66.45	59.46	54.05	56.76	52.78	33.33	43.06	42.42	10.61	26.51
	diff unit	gpt-5-mini	63.16	47.37	55.27	42.11	34.21	38.16	31.82	16.22	24.02	45.16	4.62	24.89	24.24	1.52	12.88
		Qwen3.5-122B	60.53	51.32	55.92	50.00	42.11	46.06	56.76	25.68	41.22	38.89	13.89	26.39	36.36	9.09	22.73
		Qwen3.5-35B	65.79	40.79	53.29	55.26	31.58	43.42	43.24	20.27	31.76	27.78	6.95	17.36	24.24	1.52	12.88
		Qwen3.5-9B	36.84	28.95	32.90	26.32	13.16	19.74	8.11	8.11	8.11	8.33	0.00	4.17	0.00	0.00	0.00
finance	same unit	gpt-5-mini	90.48	94.05	92.27	100.00	97.62	98.81	88.10	90.48	89.29	81.08	70.73	75.91	81.58	19.74	50.66
		Qwen3.5-122B	88.10	90.48	89.29	88.10	83.34	85.72	88.10	77.38	82.74	87.80	59.76	73.78	73.68	43.47	58.58
		Qwen3.5-35B	90.48	92.86	91.67	90.48	80.95	85.72	78.57	80.95	79.76	92.68	54.88	73.78	71.05	39.48	55.26
		Qwen3.5-9B	57.14	70.24	63.69	50.00	67.86	58.93	64.29	50.00	57.15	53.66	34.15	43.91	36.84	17.11	26.97
	diff unit	gpt-5-mini	80.95	85.72	83.33	95.24	89.29	92.26	92.86	82.14	87.50	72.97	58.54	65.75	76.32	13.16	44.74
		Qwen3.5-122B	85.71	80.95	83.33	90.48	80.95	85.72	80.95	63.10	72.02	75.61	45.13	60.37	73.68	30.26	51.97
		Qwen3.5-35B	80.95	77.38	79.17	90.48	78.57	84.53	71.43	66.67	69.05	73.17	47.56	60.37	55.26	31.58	43.42
		Qwen3.5-9B	47.62	48.81	48.22	59.52	44.05	51.78	52.38	38.10	45.24	31.71	15.85	23.78	18.42	2.63	10.53

Table 4: Performance on parallel questions across domains, unit settings, models, and numbers of tables. For each number of tables, results are reported for relational questions, quantitative questions, and their average.

		2			3			5		
		rel	quant	avg	rel	quant	avg	rel	quant	avg
environ.	gpt-5-mini	98.00	80.00	89.00	69.39	61.23	65.31	56.25	47.92	52.08
	Qwen3.5-122B	78.00	65.00	71.50	61.22	41.84	51.53	56.25	23.96	40.11
	Qwen3.5-35B	84.00	72.00	78.00	65.31	46.94	56.13	60.42	28.13	44.27
	Qwen3.5-9B	54.00	34.00	44.00	34.69	24.49	29.59	33.33	18.75	26.04
finance	gpt-5-mini	80.43	65.22	72.83	68.89	45.56	57.22	45.24	44.05	44.65
	Qwen3.5-122B	65.22	59.78	62.50	64.44	27.78	46.11	42.86	29.76	36.31
	Qwen3.5-35B	69.57	44.57	57.07	46.67	25.56	36.11	50.00	33.33	41.67
	Qwen3.5-9B	34.78	20.65	27.72	33.33	12.22	22.78	33.33	14.29	23.81

Table 5: Performance across domains, models, and numbers of tables. For each number of tables, results are reported for relational questions, quantitative questions, and their average.

effort, since the model may miss domain-relevant regularities or propose relationships that a practitioner would consider implausible. This is acceptable in our default setting, whose goal is controlled diagnostic generation rather than faithful reproduction of real sources. When higher plausibility is desired, GRADINO exposes the same constraint machinery as a user-driven mode, following the semi-automatic supervision strategy of Ahmad et al. (2025). In this mode the LLM only proposes a candidate schema (§B.1) and a candidate constraint set in the format of Figures 9 and 10. A domain expert then inspects, edits, removes, or adds rules directly in this format, which is human-readable by design: row selectors are equalities over categorical attributes, and predicates use a restricted, Python-evaluable syntax. Authoring is incremental, as the user iterates on the proposed rules rather than writing them from scratch, and any edited rule set is passed through the same parser (§B.2) and solver-backed sampler (§B.3) as the automatic pipeline. This places domain knowledge directly under user control. GRADINO builds the interface on two well-studied constraint families, namely conditional functional dependencies for intra-row rules (Rammelaere and Geerts, 2019) and denial constraints for inter-row rules (Chu et al., 2013), chosen for their expressiveness over the categorical-and-single-numeric structure we target. The interface is extensible to further families without altering the rest of the pipeline, and the Z3 encoding of §B.2 guarantees that the instantiated latent source

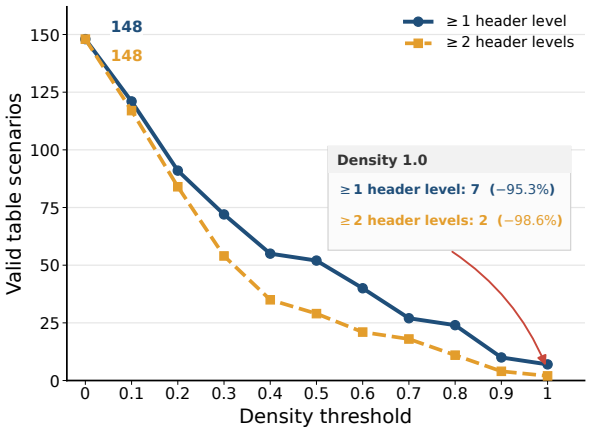


Figure 7: Feasibility of constructing dense pivoted non-relational views from existing relational tables. Dense hierarchical pivots are rare: at density 1.0, only 7 tables support at least one header level, and only 2 support at least two header levels.

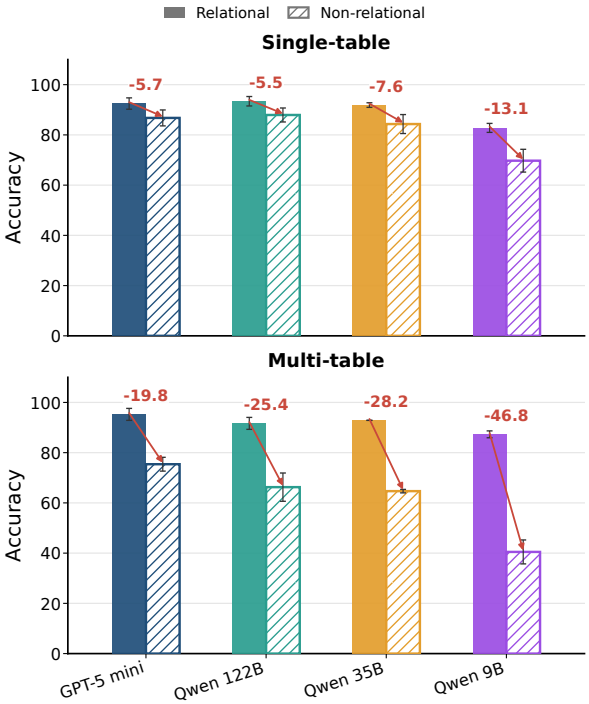


Figure 8: Performance gap between relational and quantitative parallel questions. The plot reports relational accuracy minus quantitative accuracy for each model as the number of tables increases. Positive values indicate that quantitative questions are harder than relational ones, especially when many table-specific numerical values must be extracted and aggregated.

satisfies every rule the user supplies. Constraint coverage and plausibility thus rest with the user, who is the appropriate judge of domain validity.

Semi-automatic enforcement changes only the plausibility of the latent source. Constraints are never part of the supervision signal, and gold answers are always obtained by executing the SQL programs over the instantiated tables (§3.2), so neither the choice nor the coverage of constraints can affect answer correctness or evidence preservation. The semi-automatic mode therefore lets users trade generation effort for realism along a continuous range, from fully automatic from-scratch generation to expert-validated schemas and constraints, while preserving GRADINO's executable guarantees throughout.

C Generating non-relational QA instances from real relational tables

This appendix elaborates on the two main motivations underlying GRADINO: (i) the intrinsic difficulty of synthesizing non-relational QA instances from existing relational tables, and (ii) the marked drop in LLM performance when answering questions over such non-relational tables synthesized from real ones.

Perturbed parallel and sequential questions are difficult to synthesize from real tables. Not all perturbations and numerical question types can be obtained by transforming real relational tables. To test this, we extracted databases from Spider, BIRD, and BEAVER, obtaining 2,096 relational tables, and retained only the 148 tables with at least one floating-point attribute, and satisfying tabular size constraints (more than 2 columns and 9 rows). For each table, we attempted to construct a pivoted view by selecting the attributes that maximize density, defined as the fraction of non-NaN cells in the resulting pivot. We do not search over subsets of attribute values, since this would make the pivot search combinatorial and impractical for tables with many tuples.

As shown in Figure 7, even a simple one-level row and column pivot discards 95.3% of the candidates, while requiring at least two hierarchical levels on either rows or columns discards 98.6%. Thus, common non-relational perturbations such as dense pivots and hierarchical headers are rarely directly applicable to real relational sources. The constraint becomes even stronger for parallel and sequential multi-table scenarios, which require compatible evidence values, valid lookup chains, sufficient table sizes, and, when units are perturbed, convertible numerical quantities. By generating the latent source and the perturbed views jointly, GRADINO satisfies these constraints by construction.

Non-relationalization amplifies errors in multi-table settings, even in real tables. Starting from the two relational tables that support at least two hierarchical levels on rows or columns, we extend the source pool with five additional tables from three databases crawled from Kaggle[2]: Northwind[3], Bikestores[4] and SQL Bootcamp[5]. We then use GRADINO to apply all structural and cell-level perturbations, and generate both single-table instances and 3-table parallel scenarios. This yields 108 single-table and 84 multi-table instances, repeated over three seeds for more stable estimates.

Figure 8 shows that non-relationalization consistently reduces accuracy, but the drop is much larger in the multi-table setting. In single-table instances, performance decreases by 5.7 points for `gpt-5-mini`, and by 5.5, 7.6, and 13.1 points for Qwen 122B, Qwen 35B, and Qwen 9B. In multi-table instances, the drops increase to 19.8, 25.4, 28.2, and 46.8 points. Importantly, this cannot be explained by the number of tables alone: in the clean relational setting, performance does not decrease when moving from single-table to multi-table inputs. The degradation therefore comes from the interaction between multi-table reasoning and non-relational variability across views.

D Experimental Results and Training Configuration

This appendix provides additional details on the experimental evaluation (§4.4). Specifically, it reports detailed per-setting performance of the evaluated baselines on parallel and sequential questions, over both relational and non-relational tables generated by GRADINO (§D.1), together with the configuration and results of the finetuning experiments (§D.2).

D.1 Detailed QA Performance

Table 4 reports the performance of `gpt-5-mini`, `Qwen3.5-122B`, `Qwen3.5-35B`, and `Qwen3.5-9B` on parallel questions in the environmental and financial domains, under both uniform and heterogeneous units of measurement. Consistent with §4.4, accuracy decreases as the number of tables grows (*scaling collapse*) and when compatible quantities are expressed in different units (*unit heterogeneity*).

	single-table		multi-table		
model	rel.	non-rel.	rel.	non-rel.	avg
gpt-5-mini	92.48	86.76	95.24	75.40	87.47
Qwen3.5-122B	93.40	87.93	91.66	66.27	84.81
Qwen3.5-35B	91.88	84.32	92.86	64.68	83.43
Qwen3.5-9B	82.79	69.73	87.30	40.48	70.08
avg	90.14	82.19	91.76	61.71	–

Table 6: Average performance over 3 seeds across models and table settings. Results are grouped into single-table and multi-table (3 tables) settings, with relational and non-relational questions reported separately.

Table 5 reports the corresponding results on sequential questions. Like in the parallel setting, scaling the amount of tables sharply decreases the performance on sequential questions. Although sequential questions are harder at matched table counts, they are inherently bounded by the number of tables, since realistic lookup chains rarely span more than five tables. Parallel questions, in contrast, can be made arbitrarily harder by increasing the number of tables, as their difficulty is governed by the amount of evidence that must be located and aggregated.

Table 6 reports model performance on questions generated from real tables extracted from Spider and Kaggle. In every setting, questions over non-relational tables are harder than those over their relational counterparts, and this gap is amplified in the multi-table case. While non-relational presentation reduces accuracy by 7.95 points on average in the single-table setting, the drop grows to 30.05 points in the multi-table setting, showing that table count and non-relationality interact to produce a compounded failure mode.

D.2 Training configuration

We finetune `Qwen2.5-14B-Instruct` (no-thinking) with LoRA (Hu et al., 2022) to assess whether GRADINO-generated data is useful as training supervision (§4.4). LoRA adapters are applied with r=8, α=16, no dropout and no bias to the `query` and `value` attention projections, and the model is loaded in `bfloat16` with SDPA attention. Training uses AdamW with a learning rate of 2×10^{-4}, 10 warmup steps, a maximum sequence length of 16,384 tokens, and an effective

[2] used solely for testing, without redistribution

[3] https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/northwind-pubs, MS-PL license

[4] https://www.sqlservertutorial.net/sql-server-sample-database/

[5] https://github.com/anshlambagit/SQL_Bootcamp

					Question composition (balanced)		
Experiment	Split	# inst.	% disc.	# tables	agg. op.	units	rel./non-rel.
Scaling & units (Fig. 4, 6)	parallel	2,322	22.6	$\{2, 3, 5, 10, 20\}$	sum/avg/sup.	same/mixed	—
	sequential	840	6.7	$\{2, 3, 5\}$	sum/avg/sup.	same	—
Rel. vs non-rel. (Fig. 8)	single-table	216	0.0	1	sum/avg/sup.	same	✓
	multi-table	168	22.2	3	sum/avg/sup.	same	✓

Table 7: Statistics of the GRADINO-generated data used in our experiments. **# inst.** is the number of instances retained after the question-verification pass (§3); **% disc.** is the share of generated candidates discarded by that pass. Under *Question composition*, each split is balanced across the listed table counts, aggregation operators (sum, average, superlative), unit settings (same vs. mixed units, where applicable), and—for the ablation—relational vs. non-relational views. The rel./non-rel. ablation counts both the relational and non-relational rendering of each latent instance ($216 = 108 \times 2$ single-table, $168 = 84 \times 2$ multi-table; cf. §4.4).

batch size of 4 (per-device batch size 1 with gradient accumulation 4). We train for 10 epochs under FSDP full-shard data parallelism on 4xA100 GPUs. Following common practice for instruction tuning (Wang et al., 2023), the loss is computed only over the completion tokens, ignoring the prompt. We evaluate at the end of each epoch and retain the checkpoint with the lowest validation loss. All runs use a fixed seed.

Training data. For the GRADINO regimes, training instances are drawn from the generated parallel questions over $\{2, 3, 5\}$ tables, balanced across the three aggregation operators (average, sum, and superlative) and the same-unit and mixed-unit settings. For TAT-QA, we retain only the instances answerable from tables alone, whereas for GRI-QA we partition its multi-table instances into the train and test splits used throughout our evaluation. Across all regimes, we keep only the instances whose answer, computed by `gpt-5-mini`, was verified as correct, and use the associated reasoning trace as the supervised target, teacher-forcing `Qwen2.5-14B-Instruct` on the completion only. To ensure a fair comparison across data sources, every regime is capped at the same number of training instances (440), split 90%/10% between training and validation. Models are evaluated on the GRI-QA test split (§4.4, Table 2).

D.3 Program-of-Thoughts results

Table 8 reports the results of `gpt-5-mini` with Program-of-Thoughts prompting (Chen et al., 2023). When the generated Python code is not executable, we fall back to Chain-of-Thought prompting. Although Program-of-Thoughts yields slightly higher accuracy than Chain-of-Thought in some settings (Table 4), performance still degrades sharply as the number of tables increases and under unit heterogeneity. This suggests that the observed failures are not primarily caused by incorrect arithmetic execution, but by broader difficulties in resolving the question, retrieving the relevant evidence, and handling heterogeneous table presentations.

E Statistics of GRADINO-generated data

Table 7 reports, for each experiment, the number of retained instances, the percentage of generated candidates discarded by the question-verification pass, and how the instances are balanced across table counts, aggregation operators, unit settings, and relational versus non-relational views.

Parallel questions are the most numerous, as they span more table-count settings (including the 10- and 20-table regimes) and are further split into same-unit and mixed-unit variants. They also exhibit the highest discard rate (22.6%). This is a consequence of enforcing the semantic constraints (§B): when the latent source is partitioned into per-table views for parallel questions, constraint enforcement can sharply shrink the feasible table size, leaving some candidate views too small to satisfy the layout requirements (each view is required to have between 3 and 10 rows and 3 and 10 columns). Sequential questions, by contrast, are discarded far less often (6.7%). Their losses stem mainly from hallucinations in the LLM-generated schema, such as malformed JSON or type mismatches (e.g., a categorical attribute declared as `float`).

The relational-vs-non-relational ablation (Figure 8) behaves differently, because it operates on real tables and therefore skips both schema generation and semantic-constraint enforcement. In the single-table setting no instance is discarded,

Domain	Setting	2			3			5			10			20		
		rel	quant	avg	rel	quant	avg	rel	quant	avg	rel	quant	avg	rel	quant	avg
Environ.	Same unit	97.37	88.16	**92.77**	92.11	90.79	**91.45**	91.89	90.54	**91.22**	91.67	70.83	**81.25**	84.85	24.24	**54.55**
	Unit diff.	60.53	50.00	**55.27**	50.00	38.16	**44.08**	40.54	16.22	**28.38**	22.78	4.17	**13.48**	24.24	1.52	**12.88**

Table 8: Accuracy of `gpt-5-mini` with Program-of-Thoughts prompting (Chen et al., 2023) on the environmental domain by number of input tables and unit setting.

since the tables are used as-is, although they are themselves the product of an aggressive filtering of Spider, BEAVER, and BIRD (Figure 7). The multi-table setting skips the same two stages but still requires parallel splits: a valid split must isolate the target value by varying a single attribute while keeping each view within the admissible table size range and pivot density, and such a split cannot always be found. This raises the discard rate to 22.2%, comparable to the synthetic parallel setting despite the absence of constraint enforcement.

F Prompts

This appendix collects the prompts used by GRADINO at each stage of the generation pipeline. Figure 9 shows the prompt used to generate the latent relational source (§3.1), and Figure 10 the prompt used to infer the semantic constraints (§B). Figures 11–13 report the question-generation prompts for the same-unit parallel, mixed-unit parallel, and sequential regimes, while Figures 14–16 report the corresponding question-verification prompts. Finally, Figure 17 shows the inference prompt provided to the evaluated models. All prompts instruct the model to reason in a zero-shot Chain-of-Thought (Kojima et al., 2022) manner.

G Licenses

We indicate the licenses of all the artifacts used. All the artifacts used in this paper can be used for research: Spider (CC BY-SA 4.0), BIRD (CC BY-SA 4.0), BEAVER (MIT), TAT-QA (MIT), GRI-QA (MIT), Qwen models (Apache 2.0).

Relational generation prompt

You are the best table designer in the world for the {domain} topic. You always use lexicon highly specific to {domain}. For the tables you create, you always make the tables "real", using real entities while avoiding placeholders. You always use lexicon that is specific to {domain}, and make textual entries in the tables be composed even by more words (unlike standard relational tables), like in the following examples (you must come up with different lexicon than the one used in the examples):

{examples}

You must use variations of the terminology used previously, but it's important you avoid using highly common terms. Use terms that only a real {domain} expert would know. Then, create the following Python dictionary, where the number of attributes (number of columns) must be exactly equal to num_columns.

```
{
    "name": ..., # name of the table, must be in pascal casing
    "attributes": ..., # list containing the names of the attributes. The names must be written
    ↪  in pascal casing. They must not have whitespaces
    "attributes_long": ..., # list containing the names of the attributes. This list must be
    ↪  exactly the same as the "attributes" list, but the attribute names must not be in camel
    ↪  casing, pascal casing or snake casing. They may contain consist of multiple words. Use
    ↪ {domain}-specific lexicon, as the one used in the "examples" below, but be creative and
    ↪  vary the lexicon. Not every word must have the initial letter in uppercase.
    "attribute_types": ..., \# list containing the types of the attributes. the length of this
    ↪  list must be equal to the length of the "attributes" list. The type must be
    ↪  "categorical", or either "int" or "float" for the "value_col" column. Also numerical
    ↪  values (like years or IDs) can be categorical. Only the value_col column can be float.
    "range": ..., # this list must have the same length as "attributes" and "attribute_types".
    ↪  For categorical attributes it is a list containing {col_cardinality} different values,
    ↪  which can be lengthy like in standard web tables. Use {domain}-specific lexicon like
    ↪  the one used in the examples below, but be creative and vary the lexicon. For possible
    ↪  float and int values it is a list containing, at the first position, the start of the
    ↪  range and at the second position the end of the range (extremes included).
    "value_col": ..., # string indicating the attribute name of the column to pivot later. The
    ↪  name of the attribute must be one of the names in "attributes". This table attribute
    ↪  must contain values that are either "int" or "float".
    "unit_of_measurement": ..., # one string, where the unit must be one of the following
    ↪  units: {units}
    "number_of_decimals": ... # integer value indicating the number of decimals (integer
    ↪  greater or equal than 0) to use when representing values in the "value_col" column.
}}
```

The generated table must NOT use the following attributes and values: Attributes of past tables: {past} Corresponding values of past tables: {past_values}
First reason step-by-step. Then write exactly "Final answer: " (the text must be exactly the same) followed exclusively by the requested Python dictionary. To avoid numerical inconsistencies, you must not specify any type of unit of measurement inside the table ("name", "attributes", "attributes_long", "attribute_types", "range", "value_col"), but only inside the "unit_of_measurement" field. This is very important. Ensure the output is in the expected format. Make sure that the proposed table is about {domain} and at the same time does uses completely different attributes and values as the tables used previously. Make sure that the table values ("range") do not include any specification about units of measurement, as the unit of measurement must be specified only in the "unit_of_measurement" field. This is very important to avoid numerical inconsistencies. Make sure to write exactly "Final answer: " at the end (the text, together with ":", must be exactly and completely the same), followed by the required dictionary. Do not write anything else after "Final answer: ".
Let's think step-by-step.

Figure 9: Prompt used by `gpt-5-mini` to generate the latent relational source.

Semantic constraint generation prompt

You are an Operations Research expert in the {domain} domain. Given table metadata, infer realistic constraints among attributes and values.
Input
You will receive a JSON-like object:
{schema_json}
Task
Infer constraints using domain knowledge. Produce:
- inter-row constraints: relationships between *different rows*, always comparing the numeric value in ‘value_col‘
- intra-row constraints: relationships between *attributes within the same row*
Do not create constraints that are already explicitly defined by the table structure (e.g., bounds in the range list, unit of measurement, number of decimals...).
Row selection syntax
- Select a row by fixing categorical values: Attribute == "Value" (the operator can be a python boolean operator)
- Combine selectors: (Attribute1 == "Value1" and Attribute2 == "Value2" and ...)
- Refer to the numeric value using the ‘value_col‘ name as a property: (...selectors...).{{value_col}} where {{value_col}} is the placeholder containing the value_col attribute name, not the string "value_col", and must not be used with the {{}} (which are part of the placeholder)
Example selectors:
- (Company == "Ferrari")
- (Year == 2020 and TypeOfMoney == "Gross") String values expect to be enclosed within "", while numerical data must not.
Constraint language
Write each rule as a Python-valid boolean/math expression (>, >=, ==, !=, +, -, *, /, and, or, not, parentheses).
- Inter-row constraints MUST compare ‘(...selector...).{{value_col}}‘ between two (or more) selected rows. The selector must be surrounded by brackets. Across each selected row, all non-specified attributes are considered to have the same value.
- Intra-row constraints MUST compare attributes in the same row and must use conditional form. The selector and expression must be surrounded by brackets:
"if (...selector...) then (<expression>)"
Examples (inter-row):
- (TypeOfMoney == "Gross").{{value_col}} > (TypeOfMoney == "Net").{{value_col}}
- (Year == 2020 and TypeOfMoney == "BudgetYearBeginning").{{value_col}} == (Year == 2019 and TypeOfMoney == "BudgetYearEnd").{{value_col}} (this applies to the same Company in both rows, as Company is not specified in the selectors)
Examples (intra-row):
- if (CompanyTier == "Small") then ({{value_col}} <= 5000000)
- if (TypeOfMoney == "COGS" or Scenario == "Actual") then (Scenario != "Forecast") The expression (e.g. Scenario != "Forecast") cannot contain "and", "or" or "not" operators (for the "and", multiple rules must be created, instead of using "and")
When you have lhs or rhs with mathematical operations (not boolean operations), enclose the lhs or rhs with "()" parentheses.
Output format (strict)
First, reason step-by-step. Then output exactly "Final answer:" followed by a Python dictionary in this format:

```
{
  "inter_row_constraints": [<string>, ...], # the list of strings can be empty if no inter-row
  ↪  constraints are found
  "intra_row_constraints": [<string>, ...] # the list of strings can be empty if no intra-row
  ↪  constraints are found
}
```

Make sure that you use the exact attribute names and values as provided in the metadata, and that you exactly follow the constraint language and output format. Remember that {{value_col}} is the placeholder containing the value_col attribute name, not the string "value_col", and must not be used with the {{}} (which are part of the placeholder)
Table info: {input}
Let’s think step-by-step.

Figure 10: Prompt used by `gpt-5-mini` to generate the semantic constraints.

Parallel same unit question generation prompt

Given a list of SQL queries, a list of HTML tables, a final aggregation operator across the tables, and the final result, you must generate a natural language question. In particular, to obtain the final result, each SQL query is executed on its corresponding table, and then the aggregation operator is applied to the intermediate results. The natural language question must be answerable by the same result. However, you must differ how the sentence is phrased and the words used, in order to avoid too much overlap between question and table contents. You must not use the exact SQL table attributes, where the attribute name is in camel case, but you must use them in a more natural way, resembling user questions. First reason step-by-step. Then, write exactly "Final question:" followed exclusively by the novel natural language question. Make sure to include all the information needed to answer correctly the question, so that the question is still answerable with the original SQL result. Do not write anything else after "Final question:".

{text}

The generated question must be of the following type: {method}
For superlative questions, make sure to ask for the value. Never ask for the entity.
The final aggregation operation, that is applied to the extracted values from each table, is the following: {aggregation}
This is the final result: {result}

Make sure that the generated question also explicits the final aggregation operation applied over the extracted values.
The question must be user-like: it must not contain SQL syntax, camel case, mathematical or boolean symbols, or explicit mentions of certain multi-word string values (appearing in the table) enclosed in "" (slight variations are fine).
Also, the natural language question must not miss any crucial information needed to answer the question.
Let's think step-by-step.

Figure 11: Prompt used by `gpt-5-mini` to generate the natural language question for same-unit parallel instances.

Parallel mixed unit question generation prompt

Given a list of SQL queries, a list of HTML tables, a final aggregation operator across the tables, an unit of measurement and the final result, you must generate a natural language question. In particular, to obtain the final result, each SQL query is executed on its corresponding table, the values must be normalized to the requested unit of measurement, and then the aggregation operator is applied to the intermediate results. The natural language question must be answerable by the same result. However, you must differ how the sentence is phrased and the words used, in order to avoid too much overlap between question and table contents. You must not use the exact SQL table attributes, where the attribute name is in camel case, but you must use them in a more natural way, resembling user questions.
First reason step-by-step. Then, write exactly "Final question:" followed exclusively by the novel natural language question. Make sure to include all the information needed to answer correctly the question, so that the question is still answerable with the original SQL result. Do not write anything else after "Final question:".

{text}

The generated question must be of the following type: {method}
For superlative questions, make sure to ask for the value. Never ask for the entity.
The final aggregation operation, that is applied to the extracted values from each table, is the following: {aggregation}
This is the final result: {result}
Make sure that the generated question also explicits the final aggregation operation applied over the extracted values.
Make sure that the final natural language question always specifies the following unit of measurement: {unit} The question must be user-like: it must not contain SQL syntax, camel case, mathematical or boolean symbols, or explicit mentions of certain multi-word string values (appearing in the table) enclosed in "" (slight variations are fine). Also, the natural language question must not miss any crucial information needed to answer the question.
Let's think step-by-step.

Figure 12: Prompt used by `gpt-5-mini` to generate the natural language question for mixed-unit parallel instances.

Sequential question generation prompt

You will be given some SQL queries that are applied sequentially to each table, some tables and the result of executing those SQL queries. You must generate a single natural language question that answers the final result. The question must be phrased exactly with the following style:

"What is the [operation] [attribute_value with unit of measure] for [constraints]?
Restrict the calculation to:
1) first option in last SQL query
2) second option in last SQL query
..."

In particular,
(1) the [constraints] must be the WHERE constraints appearing inside the first SQL query;
(2) the options must be the options appearing inside the last SQL query;
(3) by looking at the tables' contents, make sure that the question is semantically sound and plausible;
(4) you must differ how the sentence is phrased and the words used, in order to avoid too much overlap between question and table contents, as well as overlap with the value of the attributes;
(5) you must not use the exact SQL table attributes, where the attribute name is in camel case, but you must use them in a more natural way, resembling user questions;
(6) when possible, prefer using shorter questions: the less tokens you use the better, without losing any important details, such as the constraints or the final question. Variations in the formulation of the values of the attributes between NL and SQL questions are advised, as long as the question remains unambiguous.

Additionally:
(1) the generated question must be of the following type: {method}
(2) for superlative questions, make sure to ask for the value. Never ask for the entity.

First reason step-by-step. In the end, write "Final question:" followed exclusively by the final question. Do not write anything else after "Final question:".

{text}

This is the final result: {result}
Let's think step-by-step.

Figure 13: Prompt used by `gpt-5-mini` to generate the natural language questions for sequential instances.

Parallel same unit question verification prompt

You will be given a question in natural language, some SQL extraction queries, some tables and the result of executing those SQL queries + an aggregation operation (e.g. sum, average, max/min extraction or similar) on those tables. Your task is to verify if the question in natural language is equivalent to the aggregation of the SQL extractive queries and produces the same result. You must check if the natural language question does not specify a needed constraint that the SQL queries specifies. Also, you must check if the natural language question does not specify the final aggregation operation to apply. First, reason step-by-step and analyze the natural language question and the SQL queries + aggregation operation to determine if they are asking for the same information. If they are equivalent (use the same information), in the end write exactly "Final answer: Yes". If they are not equivalent, in the end write exactly "Final answer:" followed exclusively by the novel formulation of the natural language question that would make it equivalent to the SQL queries + aggregation operation. In this case, the natural language question must be user-like: it must not contain SQL syntax, camel case, mathematical or boolean symbols, or explicit mentions of certain multi-word string values (appearing in the table) enclosed in "" (slight variations are fine). Do not write anything else after "Final answer:".

Natural Language Question: {nl_question}
SQL Question: {sql_question}
Tables: {table}
Result: {sql_result}

Remember that, if you change the question, the natural language question must be user-like: it must not contain SQL syntax, camel case, mathematical or boolean symbols, or explicit mentions of certain multi-word string values (appearing in the table) enclosed in "" (slight variations are fine).
Let's think step-by-step.

Figure 14: Prompt used by `gpt-5-mini` to verify the natural language question for same-unit parallel instances.

Parallel mixed unit question verification prompt

You will be given a question in natural language, some SQL extraction queries, some tables and the result of executing those SQL queries + an aggregation operation (e.g. sum, average, max/min extraction or similar) on those tables. Also, you will be provided with the reference unit of measurement to use inside the question. Your task is to verify if the question in natural language is equivalent to the aggregation of the SQL extractive queries and produces the same result, considering also possible unit of measurement normalizations. You must check if the natural language question does not specify a needed constraint that the SQL queries specifies. Also, you must check if the natural language question does not specify the final aggregation operation to apply or the final unit of measurement the answer expects First, reason step-by-step and analyze the natural language question and the SQL queries + aggregation operation + unit of measurement to determine if they are asking for the same information. If they are equivalent (use the same information), in the end write exactly "Final answer: Yes". If they are not equivalent, in the end write exactly "Final answer:" followed exclusively by the novel formulation of the natural language question that would make it equivalent to the SQL queries + aggregation operation + unit of measurement. In this case, the natural language question must be user-like: it must not contain SQL syntax, camel case, mathematical or boolean symbols, or explicit mentions of certain multi-word string values (appearing in the table) enclosed in "" (slight variations are fine). The final natural language question must always specify the following unit of measurement: {unit}. Do not write anything else after "Final answer:".

Natural Language Question: {nl_question}
SQL Question: {sql_question}
Tables: {table}
Result: {sql_result}

Remember that, if you change the question, the natural language question must be user-like: it must not contain SQL syntax, camel case, mathematical or boolean symbols, or explicit mentions of certain multi-word string values (appearing in the table) enclosed in "" (slight variations are fine). Also, the final natural language question must always specify the following unit of measurement: {unit}.
Let's think step-by-step.

Figure 15: Prompt used by `gpt-5-mini` to verify the natural language question for mixed-unit parallel instances.

Sequential question verification prompt

You will be given a question in natural language, SQL queries that are applied sequentially, some tables and the result of executing those SQL queries. You must check if the natural language question does not specify a needed constraint that, without it, makes it impossible to correctly answer the question. If there are any missing constraint, your MUST explicitly add those constraints, without unnecessarily modifying other parts of the question.
The question must be formulated exactly with the following style:

"What is the [operation] [attribute_value with unit of measure] for [constraints]?
Restrict the calculation to:
1) first option in last SQL query
2) second option in last SQL query
..."

First, write your step-by-step reasoning and analyze the natural language question and the SQL queries to determine if there are any missing constraints. If they are equivalent (use the same information), at the end of your reasoning write exactly "Final answer: Yes". If they are not equivalent, at the end of your reasoning write exactly "Final answer:" followed exclusively by the novel formulation of the question with the missing constraints. Do not write anything else after "Final answer:". Slight variations in the formulation of the values of the attributes between NL and SQL questions are advised, as long as the question remains unambiguous.

Natural Language Question: {nl_question}
SQL Question: {sql_question}
Tables: {table}
Result: {sql_result}

Remember that, if you change the question, the natural language question must be user-like: it must not contain SQL syntax, camel case, mathematical or boolean symbols, or explicit mentions of certain multi-word string values (appearing in the table) enclosed in "" (slight variations are fine). The final question must always clearly explicitate what are the values that are needed to apply the aggregation operation in the end. It must not say "what is the value across the specified indicators", but say something among the lines of "what is the value across: (1) ..., (n) ...". This is important, as the user cannot see the SQL query, and everything needs to be explicitly stated. Also, the final natural language question must always specify the following unit of measurement: {unit} Remember also to first reason step-by-step. In the end, write "Final answer:" followed exclusively by the answer. Do not write anything else after "Final answer:".
Let's think step-by-step.

Figure 16: Prompt used by `gpt-5-mini` to verify the natural language question for sequential instances.

Inference prompt

Answer the following question given the provided HTML table.
First reason step-by-step, then write "Final answer:" followed exclusively by the correct answer. Do not write anything else after "Final answer:"
Every calculation must be done with a precision of exactly 6 decimal places.
Only the numerical value must be written in the final answer.

Question: {question}
Table: {table}

Let's think step-by-step.

Figure 17: Inference prompt. The prompt asks for a broad numerical precision of 6 decimals values, as the evaluation script will compute the exact match with the required precision between predicted and gold answers.